

UE5 클라이언트 개발자 포트폴리오

RulesHorror

UE5 게임 클라이언트 시스템 프로토타입
Windows · C++ · UMG/Slate · Enhanced Input

문서 목적

기능 목록을 나열하기보다, 프로젝트를 만들면서 어떤 구조를 설계했고 어떤 기술 문제를 어떤 경계에서 해결했는지 설명합니다.

장윤제
jjw1270@gmail.com
github.com/jjw1270/RulesHorror

목차

문서형 포트폴리오로 읽히도록 흐름을 기능 소개 → 설계 → 구현 근거 → 남은 과제로 정리했습니다.

01

프로젝트 개요와 현재 범위

현재 구현된 시스템 기반과 남은 클라이언트 작업 범위

02

전체 아키텍처

게임 본체와 기능 플러그인의 책임 분리

03

디제틱 컴퓨터 / 모니터 UI

월드 Actor, Pawn, WidgetInteractionComponent 연결

04

3D 모니터 입력 보정

월드 Hit를 UMG 입력 좌표로 변환하는 방식

05

StoryFlow 설계

Scene / Shot / Branch / Transition 기반 진행 시스템

06

StoryFlow 런타임과 에디터

Compile, 검증, 레벨 전이, 저장/복원 경계

07

Data / Save / Validation

Registry, DataTable, SaveGame, 비동기 수정 잠금

08

자동화와 다음 작업

동기화·빌드 자동화, 현재 성과와 남은 리스크

프로젝트 개요와 현재 구현 범위

“완성 게임 소개”가 아니라, 검토 가능한 클라이언트 시스템 구현 범위를 기준으로 정리했습니다.

프로젝트 한 줄 요약

UE5 기반 1인칭 호러 게임 클라이언트 프로토타입입니다. 현재 포트폴리오에서는 콘텐츠 서사보다 상호작용, 디제틱 모니터 UI, 그래프 기반 진행 시스템, 데이터/저장 검증 구조를 검토 대상으로 둡니다.

구분	현재 구현된 기반	아직 필요한 작업
진행	인트로 / 새 게임 / 이어하기 / 시작 Scene 진입 흐름	후속 진행 Scene 확장과 완료 조건 연결
상호작용	InteractionSystem, Computer Actor, MovePoint 진입	허브 조사 오브젝트, 진행 조건 연결 강화
모니터 UI	3D 표면 입력, 커서, 멀티 윈도우, 콘텐츠 UI	진행 상태와 선택 가능 상태 표시
데이터/저장	Item Registry, Content Row, FStoryFlowRef 저장 기반	진행 조건과 Save Key 확장

개발 판단

미니게임 개별 완성보다 전체 진행 루프와 저장/복원 경계를 먼저 정리하는 방향으로 우선 순위를 잡았습니다.

포트폴리오 강조점

콘텐츠 서사보다 시스템 경계, 입력 처리, 그래프 진행, 검증 파이프라인을 중심으로 설명합니다.

전체 아키텍처: 게임 본체는 조립부, 기능은 플러그인 경계

프로젝트 전용 흐름과 재사용 가능한 기반을 나눠, 기능 추가 시 수정 지점을 예측할 수 있게 했습니다.

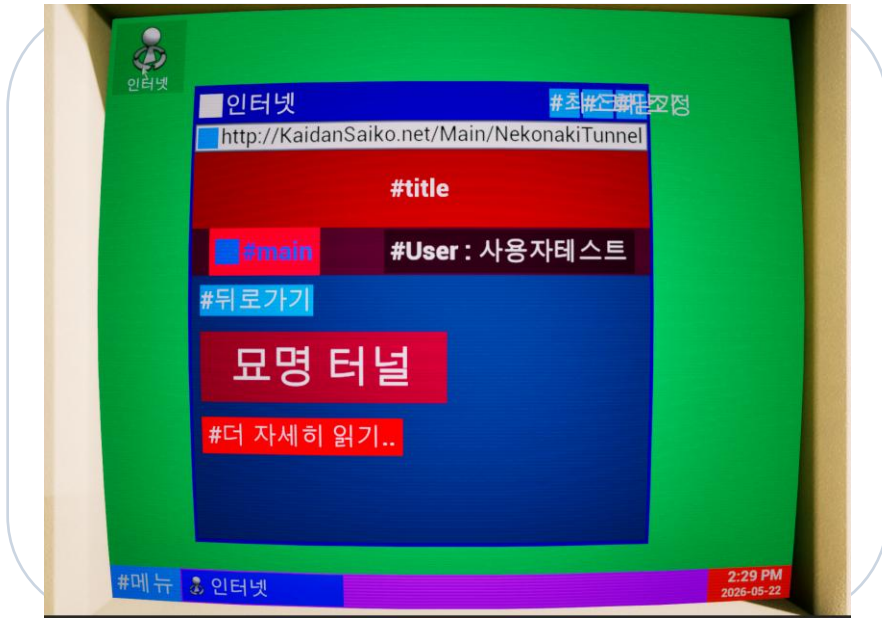


핵심 설계 기준

게임 본체는 프로젝트 맥락을 조립하고, 반복 가능한 기능은 플러그인/Subsystem으로 분리합니다. 덕분에 모니터 UI, StoryFlow, Data/Save가 서로 직접 얹히지 않고 경계 단위로 확장됩니다.

디제틱 컴퓨터 / 모니터 UI 구현

게임 외부 메뉴가 아니라, 월드에 배치된 컴퓨터 Actor를 통해 모니터 UI로 진입하는 구조입니다.



원본 캡처: 모니터 내부 UI

관련 구현

```
Computer.cpp
- Interact_Implementation()
- OpenInteractingWidget()
OfficePawn.cpp
- DriveWidgetInteraction()
```

감지

InteractorComponent
Targeted 상태

상호작용

AComputer::Interact
MovePoint 지정

진입

OfficePawn 이동 완료
전용 UI Open

조작

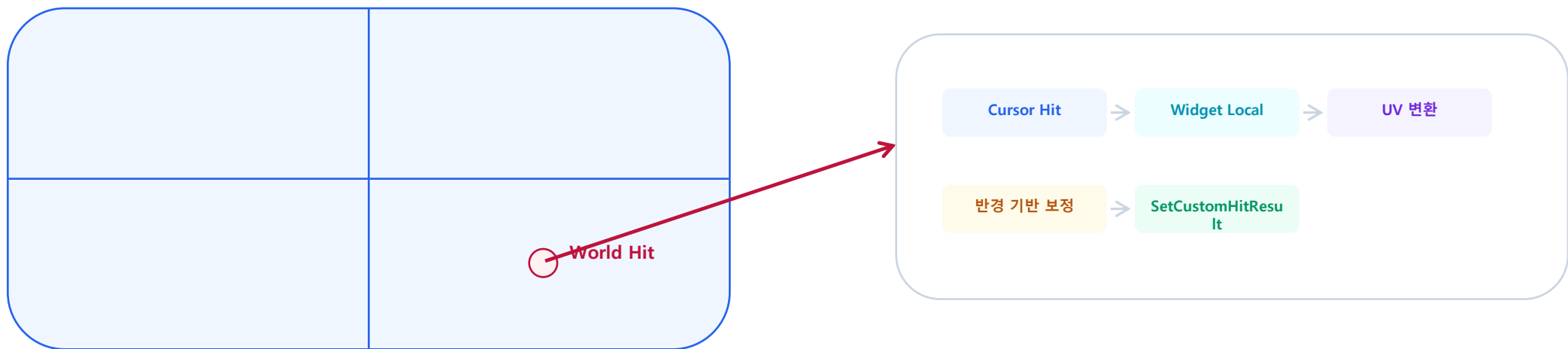
WidgetInteraction
UMG 입력 전달

역할 분리

AComputer는 장치 상태(전원, WidgetComponent, 상호작용 UI)를 관리하고, OfficePawn은 입력 전달과 모니터 조작 모드를 담당합니다. 상호작용 즉시 UI를 띄우지 않고 "접근 → 도착 → 조작" 순서로 입력 책임과 장치 상태를 분리했습니다.

3D 모니터 표면 Hit를 UMG 입력으로 보정

핵심은 UI를 예쁘게 만드는 것이 아니라, 월드 좌표와 위젯 좌표의 차이를 입력 경로에서 안정화하는 것입니다.



왜 직접 보정했나

모니터 외곽에서 클릭 위치와 커서 피드백이 어긋났고, hover / click / scroll 입력 경로를 하나의 기준으로 통일할 필요가 있었습니다.

구현 방식

PlayerController의 cursor hit를 받아 WidgetComponent 로컬 좌표와 UV로 변환한 뒤, 중심에서 멀어질수록 축별 correction scale을 적용했습니다.

결과와 한계

UMG 입력과 모니터 내부 커서가 같은 Hit 기준으로 동작합니다. 단, Mesh나 DrawSize가 바뀌면 보정 강도는 다시 튜닝해야 합니다.

핵심 코드 흐름

```
GetHitResultUnderCursorByChannel(...) → BuildCorrectedMonitorWidgetHit(hit) → WidgetInteractionComponent->SetCustomHitResult(hit)
```

StoryFlow 설계: Scene / Shot / Branch로 진행을 모델링

진행 흐름을 GameMode나 Level Blueprint 분기 나열로 만들지 않고, 편집 가능한 그래프 자산과 런타임 링크로 분리했습니다.



Story Scene Editor 원본 캡처

개념	역할
Scene	장소/상황 단위 컨테이너. StorySceneAsset 하나가 Scene 하나를 나타냄
Shot	실제 진행을 수행하는 최소 실행 단계. Enter/Tick/Finish/Exit 흐름
Branch	조건에 따라 다음 링크를 선택하는 순간 판단 노드
Transition	다음 SceneID로 이동하는 전이 노드. 런타임 객체는 만들지 않음
FlowRef	저장/복원 기준. SceneID + ShotID

왜 만들었나

컷신, 조건 분기, 레벨 이동, 저장/복원 지점이 늘어날수록 코드 분기만으로는 제작 흐름과 검증 기준이 불명확해집니다.

핵심 판단

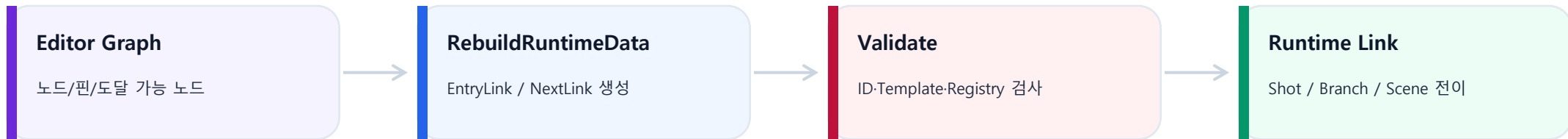
그래프는 사람이 편집하기 위한 표현이고, 런타임은 Compile된 링크 데이터만 소비하도록 분리했습니다.

저장 정책

Branch는 순간 판단이므로 저장 대상이 아닙니다. 확정된 Shot 기준으로만 복원해 상태 경계를 단순화했습니다.

StoryFlow 런타임 / 에디터 / 저장 경계

에디터 그래프를 런타임 데이터로 Compile하고, 실행 중에는 Shot / Branch / Scene 링크만 처리합니다.



시작 / 레벨 전이

StartFromScene 또는 StartFromRef로 시작합니다.
TargetLevel이 다르면 LoadingLevel을 거쳐 async load를 수행하고, PendingStartRef로 시작 위치를 보존합니다.

Shot 실행

ShotTemplate을 그대로 실행하지 않고 DuplicateObject로 런타임 인스턴스를 만든 뒤 EnterShot, TickShot, FinishShot, ExitShot 순서로 처리합니다.

Branch / Transition

BranchTemplate의 Outputs와 SelectNextIndex 결과로 다음 링크를 고릅니다. Transition은 NextSceneID 링크로 축약되어 StartFromScene으로 재진입합니다.

런타임 처리 요약

```

StartFromScene(SceneID)
  → Resolve Registry
  → Load TargetLevel if needed
  → EnterScene
  → MoveToLink(EntryLink)
  → MoveToShot / EvaluateBranch / StartFromScene
  
```

에디터 검증 항목

```

SceneID / TargetLevel / SceneTemplate
ShotID 중복 / ShotTemplate 누락
Branch Outputs / 출력 연결 누락
Transition NextSceneID Registry 등록 여부
  
```


DataTable, Registry, SaveGame, 자동화

콘텐츠 확장과 저장 안정성은 런타임에서 터지기 전에 Registry / Save / Batch 경계에서 확인하도록 배치했습니다.



Registry 검증

invalid ItemID, 중복 ID, 지원하지 않는 RowStruct, Registry DataAsset 로드 실패를 Refresh/Message Log 단계에서 드러냅니다.

SaveGame 안전성

Async save/load 중 SaveGameCanModify=false로 잠그고, Load 실패 시 기존 메모리 객체를 유지해 진행 상태 유실 전파를 막습니다.

콘텐츠 확장성

목록/상세/맵 이동은 ContentID 하나로 연결됩니다. 항목 추가는 UI 코드 수정이 아니라 Row/Registry 갱신 중심입니다.

자동화 파일	역할	안전장치
SyncBranch.bat	Git fetch/pull, Subtree, Submodule 동기화	충돌 상태면 pull 강행 금지
GenerateProjectFiles.bat	UE 엔진 경로 확인 후 Visual Studio 프로젝트 파일 생성	UE_EngineDir.txt 캐시 후 경로 재검증
BuildAndLaunch.bat	동기화 → 프로젝트 파일 생성 → Editor 빌드 → uproject 실행	단계별 errorlevel 확인 후 실패 종료

정리: 보여주고 싶은 역량과 다음 작업

현재 포트폴리오의 핵심은 화면 완성도가 아니라, 시스템을 나누고 연결하고 검증 가능하게 만든 과정입니다.

항목	보여주는 역량	대표 구현
시스템 분해	Plugin / Subsystem / Interface 경계 설정	InteractionSystem, StoryFlow, ItemCore, SaveGame
입력 처리	3D 월드 좌표와 2D UMG 좌표 연결	OfficePawn + WidgetInteraction Custom Hit
진행 구조	진행 흐름을 그래프 자산과 런타임 링크로 분리	StorySceneAsset, Shot, Branch, FlowRef
제작 안정성	런타임 전 검증과 반복 작업 자동화	Compile Validate, Registry Refresh, Batch scripts

개발하면서 얻은 결과

상호작용, 모니터 UI, StoryFlow, 데이터/저장 구조가 서로 직접 얹히지 않고 각자의 책임 경계로 연결됩니다. 검토자는 기능 화면뿐 아니라 코드와 문서에서 설계 의도를 추적할 수 있습니다.

남은 개선 방향

진행 조건 Save Key 확정, 후속 Scene 제작, 콘텐츠 선택 가능 상태 UI 반영, 반복 연출/상호작용 공통 Shot 확장이 다음 우선순위입니다.